Ajuste incorporado

O catálogo de módulos foi remodelado para utilizar o código numérico real do módulo como identificador canônico:

```
id             = 5
module_sigla   = SIGAFAT
description    = Vendas e Faturamento
```

Isso substitui o desenho atual, em que public.protheus_modules_master.id é UUID e o código numérico está separado em mod_code; atualmente, public.tenant_module_contracts.module_id também referencia um UUID. [schemas_tables | Excel]

A decisão permanece compatível com a separação conceitual documentada em AGENTS.md: domínio e módulo são entidades diferentes, e SIGAFAT corresponde ao código real 5. [onedrive.live.com]

Pacote gerado

Baixar pacote de mapeamento e migrations

O ZIP contém:

```
copilotprotheus_novo_modelo_20260930/
├── README.md
├── MAPEAMENTO_ARQUIVOS_NOVO_MODELO.md
├── manifest.json
├── VALIDACAO_ESTATICA.json
├── database/
│   └── migrations/
│       ├── 20260930_001_module_catalog_numeric_up.sql
│       ├── 20260930_001_module_catalog_numeric_down.sql
│       ├── 20260930_002_tenant_schema_factory_up.sql
│       ├── 20260930_002_tenant_schema_factory_down.sql
│       ├── 20260930_003_migrate_tenant_metadata_up.sql
│       ├── 20260930_003_migrate_tenant_metadata_down.sql
│       └── 20260930_004_post_migration_verification.sql
└── scripts/
    └── map_new_model_references.sh
```

O que as migrations fazem

Migration 001 — catálogo numérico de módulos

Cria o novo catálogo canônico:

```
CREATE TABLE public.protheus_modules_master_v2 (
    id integer PRIMARY KEY,
    module_sigla varchar(32) NOT NULL,
    description varchar(255) NOT NULL,
    active boolean NOT NULL DEFAULT true,
    created_at timestamptz NOT NULL DEFAULT now(),
    updated_at timestamptz NOT NULL DEFAULT now()
);
```

Também:

- migra mod_code para id;
- migra mod_sigla para module_sigla;
- usa mod_name ou description como descrição;
- garante o registro 5 / SIGAFAT / Vendas e Faturamento;
- converte tenant_module_contracts.module_id de UUID para inteiro;
- valida contratos órfãos;
- aborta se existirem códigos ou siglas duplicados;
- aborta caso existam FKs adicionais não mapeadas;
- preserva a tabela UUID como protheus_modules_master_legacy_uuid;
- cria uma view temporária de compatibilidade;
- possui migration de rollback.

Essa mudança é necessária porque o inventário confirma que o catálogo atual usa simultaneamente UUID, mod_code, mod_sigla e mod_name, enquanto os contratos usam o UUID. [schemas_tables | Excel]

Migration 002 — factory de schema por tenant

Instala:

```
public.provision_tenant_schema(tenant_code)
```

A função lê o schema_name de public.tenant, valida o nome e cria exclusivamente estruturas de configuração/metadados:

```
tenant_modules
dictionary_tables
dictionary_fields
dictionary_indexes
```

```
business_rules
documents
document_chunks
memories
conversations
 messages
 analyses
 dashboards
```

Não são criadas tabelas de dados transacionais do Protheus.

Também não são criadas:

```
tenant_table_permissions
tenant_field_permissions
```

O acesso a tabelas e campos passa a resultar do conteúdo efetivamente provisionado em dictionary_tables e dictionary_fields, mas a proteção de SQL, operação, timeout, volume e contexto autenticado continua obrigatória no backend.

A tabela analyses foi incluída como entidade central entre mensagens e dashboards, atendendo ao requisito de um único contexto analítico persistido para chat e dashboard definido em AGENTS.md. [onedrive.live.com]

Migration 003 — migração controlada de metadados

Instala:

```
public.migrate_legacy_tenant_metadata(
    tenant_code,
    source_schema
)
```

A função:

- provisiona o schema de destino;
- copia os módulos contratados;
- copia dictionary_tables;
- copia dictionary_fields;
- converte regras TES em regras canônicas JSON;
- mantém a origem intacta;
- filtra estruturas compartilhadas por tenant_id;
- recusa executar quando origem e destino são o mesmo schema;
- não copia tabelas transacionais do ERP.

A origem não é apagada automaticamente. Isso evita perda de informação durante o período de compatibilidade.

Migration 004 — verificação pós-migração

Executa apenas consultas de leitura que verificam:

- presença de 5 / SIGAFAT / Vendas e Faturamento;
- contratos de módulo órfãos;
- tenants ativos sem schema válido;
- FKs apontando para o catálogo de módulos;
- permanência esperada das estruturas legadas durante o período de transição.

Arquivos mapeados para alteração

A documentação recuperada confirma a existência das seguintes áreas e arquivos prioritários. [onedrive.live.com], [onedrive.live.com]

Banco e arquitetura

Alterar

```
.agents/AGENTS.md
database/schema_v5_multitenant.sql
database/schema_v2_multitenancy.sql
database/schema_knowledge_base.sql
```

A alteração deve refletir:

- módulo identificado por id integer;
- module_sigla;
- description;
- schema físico por tenant;
- AnalysisContext dentro do schema do tenant;
- ausência de permissões redundantes de tabela/campo;
- catálogo público separado do catálogo empresarial.

Não editar migrations históricas

Os seguintes arquivos foram identificados como migrations já existentes e devem ser tratados como histórico, não reescritos:

```
database/migrations/001_protheus_core_tables.sql
database/migrations/20260819_module_catalog.sql
database/migrations/20260829_dictionary_scope_cleanup.sql
database/migrations/20260829_global_catalog_metadata.sql
```

```
5
database/migrations/20260830_remove_dictionary_environment_scope.s
ql
6
database/migrations/20260831_tenant_dictionary_schema_compatibilit
y.sql
```

Esses arquivos estão documentados em Resumo executivo.md. [onedrive.live.com], [onedrive.live.com]

Painel administrativo

Alterar

```
1  admin-frontend/src/services/companyModulesApi.js
2  admin-frontend/src/pages/Companies.jsx
3  admin-frontend/src/pages/CompanyDashboard.jsx
4  admin-frontend/src/pages/Tenants.jsx
5  admin-frontend/src/pages/Licenses.jsx
6  admin-frontend/src/pages/SnapshotsPage.jsx
7  admin-frontend/src/pages/Tables.jsx
8  admin-frontend/src/pages/DictionarySyncPage.jsx
9  admin-frontend/src/pages/QueryGuardPage.jsx
10  admin-frontend/src/pages/company/Dictionary.jsx
11  admin-frontend/src/pages/governance/BusinessRulesPage.jsx
12  admin-frontend/src/pages/governance/DomainsPage.jsx
13  admin-frontend/src/pages/governance/IntentsPage.jsx
14  admin-frontend/src/pages/governance/SemanticDictionaryPage.jsx
15  admin-frontend/src/pages/governance/DashboardsPage.jsx
```

Principais mudanças de contrato:

```
1  {
2    "moduleId": 5,
3    "moduleSigla": "SIGAFAT",
4    "description": "Vendas e Faturamento"
5  }
```

O frontend não deverá mais enviar ou esperar UUID para módulo.

Candidatos à exclusão ou substituição

```
1  admin-frontend/src/pages/PermissionEditorPage.jsx
```

Deve ser excluído somente se a leitura do código confirmar que sua única responsabilidade é editar permissões redundantes de tabela/campo.

```
1  admin-frontend/src/pages/AgentRoles.jsx
```

Não deve ser excluído automaticamente. Pode continuar sendo necessário para papéis funcionais da aplicação, mesmo depois da remoção das permissões físicas de tabela e campo.

Estruturas candidatas à descontinuação

O inventário confirma a presença das seguintes estruturas atuais. [schemas_tables | Excel]

```
rodol_prd.tenant_table_permissions
rodol_prd.tenant_field_permissions
rodol_prd.company_info
rodol_prd.business_field_aliases
public.business_rule_tes
public.business_cfop_rules
public.business_financial_natures
public.copilot_conversations
public.copilot_messages
 public.copilot_dashboards
```

O pacote deliberadamente **não executa** DROP TABLE **nem** DROP SCHEMA **nas migrations UP**.

A exclusão deve acontecer somente depois de:

1. migração de todos os tenants;
2. alteração do backend;
3. alteração do frontend;
4. validação de contagem e conteúdo;
5. teste de isolamento entre tenants;
6. cutover integral das leituras e escritas;
7. período de compatibilidade;
8. backup e teste de restauração.

Backend: mapeamento pendente de conteúdo real

Os documentos confirmam a existência das áreas backend/, app/, database/, db/, migrations/, tests/, middleware/ e config/, mas o conteúdo completo dos models, services e routers relacionados não ficou acessível nesta execução. [onedrive.live.com], [onedrive.live.com]

Por isso, não inventei nomes de arquivos backend.

Incluí no pacote:

```
scripts/map_new_model_references.sh
```

Ele procura no repositório por:

```
protheus_modules_master
mod_code
mod_sigla
mod_name
tenant_module_contracts
module_id
tenant_table_permissions
tenant_field_permissions
copilot_conversations
 copilot_messages
 copilot_dashboards
 business_rule_tes
 business_cfop_rules
 business_financial_natures
 rodol_prd
 schema_name
 search_path
```

O resultado será gravado em:

```
new-model-reference-map.txt
```

Esse inventário permitirá identificar com precisão:

- models SQLAlchemy;
- schemas Pydantic;
- routers FastAPI;
- services;
- repositories;
- contratos HTTP;
- testes;
- seeds;
- scripts de provisionamento;
- dependências frontend/backend.

Validação realizada

O pacote passou por duas verificações estáticas automáticas:

- presença do seed canônico de SIGAFAT;
- migração UUID → inteiro nos contratos;
- preservação do catálogo legado;
- ausência de criação de permissões de tabela/campo;
- presença de analyses;
- vínculo obrigatório entre dashboard e análise;
- idempotência das chaves com empresa opcional;
- ausência de DROP TABLE e DROP SCHEMA nas migrations UP;
- equilíbrio dos blocos BEGIN/COMMIT.

Importante: as migrations não foram executadas contra uma instância PostgreSQL real. Portanto, antes da aplicação em produção, ainda é obrigatório validá-las em uma restauração isolada do banco real.

Também não localizei, nas buscas disponíveis, ESPECIFICACAO_TECNICA_COLABORACAO_IA_COPILOT_PROTHEUS.md nem PROJECT_MEMORY.md. Por segurança, essa pendência está registrada no pacote e impede considerar as migrations prontas para aplicação direta em produção.